

LESSON 5

Sorting-Based Problem Solving

Sorting as preprocessing turns $O(n^2)$ into $O(n \log n)$.

5.1 Python Sort Mastery

5.2 Merge Intervals

5.3 Find Duplicates

5.4 Kth Largest/Smallest

5.5 Meeting Rooms

5.6 Practice Problems

5.1 Python Sort Mastery

```
# In-place sort vs new sorted list
nums = [3, 1, 4, 1, 5]
nums.sort()           # modifies nums -> [1, 1, 3, 4, 5]
sorted_nums = sorted(nums) # new list, original unchanged

# Descending
nums.sort(reverse=True) # [5, 4, 3, 1, 1]

# Custom sort key
words = ["banana", "pie", "apple"]
words.sort(key=len)      # by length: ["pie", "apple", "banana"]

# Sort tuples by specific element
pairs = [(2, "b"), (1, "a"), (3, "c")]
pairs.sort()              # by first: [(1, "a"), (2, "b"), (3, "c")]
pairs.sort(key=lambda x: x[1]) # by second element

# Multiple criteria: grade desc, name asc
students = [("Alice", 90), ("Bob", 85), ("Carol", 90)]
students.sort(key=lambda x: (-x[1], x[0]))
# [("Alice", 90), ("Carol", 90), ("Bob", 85)]
# Negative grade = descending; name = ascending
```

5.2 Merge Overlapping Intervals (Very Popular)

```
# Input:  [[1,3], [2,6], [8,10], [15,18]]
# Output: [[1,6], [8,10], [15,18]]

def merge_intervals(intervals):
    intervals.sort(key=lambda x: x[0]) # sort by start time
    merged = [intervals[0]]

    for start, end in intervals[1:]:
        last_end = merged[-1][1]
        if start <= last_end:           # overlap!
            merged[-1][1] = max(last_end, end) # extend
        else:
            merged.append([start, end]) # no overlap, add new

    return merged
```

```
Sorted: [[1,3], [2,6], [8,10], [15,18]]
merged = [[1,3]]
```

```
[2,6]: 2 <= 3? YES -> merge -> [[1,6]]
[8,10]: 8 <= 6? NO -> append -> [[1,6], [8,10]]
[15,18]: 15 <= 10? NO -> append -> [[1,6], [8,10], [15,18]]
```

5.3 Find Duplicates & Kth Largest

```

# Contains Duplicate
def has_duplicate(nums):
    return len(nums) != len(set(nums))

# Find all duplicates
def find_duplicates(nums):
    count = {}
    for num in nums:
        count[num] = count.get(num, 0) + 1
    return [num for num, c in count.items() if c > 1]

# Kth Largest Element
def kth_largest(nums, k):
    nums.sort(reverse=True)
    return nums[k-1]

kth_largest([3,2,1,5,6,4], 2) # 5 (2nd largest)

# Better: use heap for O(n log k)
import heapq
def kth_largest(nums, k):
    return heapq.nlargest(k, nums)[-1]

```

5.4 Meeting Rooms

```

# Can a person attend all meetings? (no overlap)
def can_attend(meetings):
    meetings.sort(key=lambda x: x[0])
    for i in range(1, len(meetings)):
        if meetings[i][0] < meetings[i-1][1]: # overlap
            return False
    return True

# can_attend([[0,30],[5,10],[15,20]]) # False (0-30 overlaps 5-10)
# can_attend([[7,10],[2,4]])          # True (no overlap)

```